

BCA152 Laboratory Activity No. 1

Real-Time Multisensor Room Monitoring System — Laboratory Report

Mindanao State University — Iligan Institute of Technology
College of Computer Studies — Department of Computer Applications

1. Problem and Requirements

The laboratory required the design of a simulated ESP32-based room-monitoring system, reconstructed from a stated set of functional requirements and implemented as a concurrent embedded system using ESP-IDF and native FreeRTOS APIs on PlatformIO, simulated in Wokwi. The device monitors temperature, humidity, and ambient light; detects occupancy via a PIR sensor; lets the user cycle between measurements on an OLED display using a rotary encoder; and raises a buzzer alarm when temperature falls outside a configured 18–30°C range. The system also manages an ACTIVE/INACTIVE power state, returning to a low-activity mode after 15 seconds without detected motion and reactivating immediately on the next motion event.

Because the target hardware is simulated rather than physical, the DHT22 driver, SSD1306/OLED output, and encoder handling were adapted to timings and behaviors that Wokwi's simulated peripherals support, while preserving the functional intent of each requirement (FR-01 through FR-10) listed in the assignment specification.

2. System Architecture and Design

Hardware architecture. The ESP32 interfaces with a DHT22 (temperature/humidity, GPIO4), a photoresistor read through an ADC channel (light level, GPIO34), a PIR sensor (motion, GPIO27), a KY-040 rotary encoder (navigation, GPIO18/19/5), an SSD1306 OLED over I2C (GPIO21/22), and an active buzzer (GPIO25) for the alarm.

Software architecture. The firmware is organized into one module per subsystem (sensors, display, input, alarm, motion, system_state) plus a shared rtos_objects module that centralizes every FreeRTOS synchronization primitive used across tasks. main.c performs hardware initialization, creates the shared RTOS objects, creates all tasks, and then hands control to the scheduler — it contains no application logic of its own.

Subsystem decomposition. Each subsystem owns exactly one hardware resource (e.g. only DisplayTask writes to the OLED), and cross-subsystem communication happens exclusively through queues, an event group, or task notifications — never through unsynchronized global state.

State machine. A two-state ACTIVE/INACTIVE machine is centralized in StateTask. It transitions to INACTIVE only from ACTIVE after the configured inactivity timeout elapses with no motion observed, and returns to ACTIVE immediately on any detected motion, regardless of current state.

3. FreeRTOS Architecture

Six tasks were created, each with a single clear responsibility and an explicitly justified priority. Priorities reflect scheduling urgency rather than perceived importance: MotionTask and InputTask (priority 3) must respond to real-world stimuli (a person entering the room, a knob turn) with minimal latency, since a missed or delayed motion edge could cause an incorrect ACTIVE/INACTIVE transition, and a delayed encoder read makes navigation feel unresponsive. SensorTask, AlarmTask, and StateTask (priority 2) can tolerate brief scheduling latency — a temperature sample arriving a few milliseconds late has no functional consequence. DisplayTask (priority 1) is lowest because a briefly stale OLED frame is imperceptible and never blocks a safety-relevant decision.

Task	Responsibility	Period / Trigger	Priority	IPC	Typical Blocked Co
SensorTask	Read DHT22 + LDR	2 s (vTaskDelayUntil)	2	Sends to sensor queue	Delay
DisplayTask	Own & update OLED	Periodic / on data	1	Reads sensor + page queue	Waiting for data
InputTask	Encoder navigation	Event-driven (ISR)	3	Task notification	Waiting for notification
MotionTask	PIR monitoring	Short poll	3	Sets event group bits	Delay
AlarmTask	Evaluate & drive buzzer	On sensor update	2	Reads sensor queue, sets event group	Waiting for data
StateTask	ACTIVE/INACTIVE state	Periodic	2	Reads/sets event group	Delay

Communication and synchronization. A FreeRTOS queue carries sensor readings from SensorTask to its consumers; because a plain queue is single-consumer, sensor data is fanned out via two queues (one per consumer task) rather than one shared queue, avoiding silent data starvation between DisplayTask and AlarmTask. A mutex protects shared UART/console output so diagnostic lines from concurrent tasks are never interleaved mid-line. An event group carries cross-cutting system state (ACTIVE/INACTIVE, motion-detected pulses, alarm-active) that multiple tasks need to observe without direct coupling. A task notification is used for the encoder ISR-to-task handoff, since it is a lighter-weight mechanism than a queue for a single integer value delivered from an interrupt context.

Periodic execution. SensorTask uses vTaskDelayUntil() rather than vTaskDelay() so its 2-second sampling period stays fixed relative to a running reference tick, regardless of how long the DHT22 bit-banded read takes on a given cycle. vTaskDelay() measures its delay from the moment it is called, so any variation in the preceding work accumulates as drift in the sampling period over time; vTaskDelayUntil() instead measures from the fixed last wake time, preventing that drift.

4. Implementation

Sensor acquisition and display navigation logic were deliberately separated from their hardware drivers: evaluateTemperature() and evaluateSystemState() are pure functions taking only primitive inputs and returning an enum, with no GPIO, I2C, or FreeRTOS calls inside them. This was done specifically so they could be exercised by unit tests without requiring the ESP32 target or a live simulation. Similarly, nextDisplayMode()/previousDisplayMode() implement the encoder's forward/reverse traversal, including wraparound at both ends of the DisplayMode enum, independently of how a raw encoder pulse is decoded into a direction.

A significant implementation decision was moving from a C++/ESP-IDF combination (initial scaffold) to a plain C implementation for the final build, after encountering CMake/toolchain friction specific to the

development environment (see Engineering Discussion, Section 7). The plain-C version preserves the same task/queue/mutex/event-group architecture and was the version verified running in the Wokwi simulator.

5. Verification and Testing

5.1 Unit testing.

Thirteen Unity unit tests were written against the pure decision-logic functions, meeting the minimum required coverage:

Category	Cases	Coverage
Temperature alarm logic	5	Below lower threshold; exactly at lower threshold; normal value; exactly at upper threshold; above
Display navigation	4	Forward step; forward wraparound; reverse step; reverse wraparound
System state	4	ACTIVE/no timeout; ACTIVE/timeout elapsed; INACTIVE/no motion; INACTIVE/motion

The test suite is committed at `test/test_logic/test_main.c`. Automated execution via `pico test` encountered environment-specific linker conflicts under the available PlatformIO/ESP-IDF toolchain configuration (duplicate symbol definitions arising from the interaction between the test file's direct source includes and PlatformIO's automatic source-file scanning across two build environments) that were not resolved before the submission deadline. Each test's expected value was manually traced against the corresponding function implementation to confirm correctness independent of the automated runner.

5.2 Wokwi functional verification.

The firmware was built and run in the Wokwi simulator connected to the project's `diagram.json` circuit. SensorTask output was confirmed streaming live in the serial monitor, showing periodic temperature, humidity, and light readings consistent with the configured 2-second sampling period.

Test ID	Stimulus	Expected Result	Observed Result
FT-01	Change temperature (Wokwi slider)	Displayed/logged temperature updated	PASS — SensorTask log reflects new value
FT-02	Change humidity	Displayed/logged humidity updated	PASS — confirmed in serial output
FT-03	Change light input	Light value changes	PASS — confirmed in serial output
FT-04	Rotate encoder clockwise	Next display page selected	See note below
FT-05	Rotate encoder counterclockwise	Previous display page selected	See note below
FT-06	Set temperature above 30°C	Alarm activates	See note below
FT-07	Return temperature to normal	Alarm stops	See note below
FT-08	Trigger PIR	System is ACTIVE	See note below
FT-09	Allow inactivity timeout	System becomes INACTIVE	See note below
FT-10	Trigger PIR while INACTIVE	System returns to ACTIVE	See note below

Note: FT-01 through FT-03 were directly observed via SensorTask's serial output during the simulation session captured for this submission. FT-04 through FT-10 exercise the encoder, alarm, and motion/state subsystems and are architecturally supported by the implementation (InputTask/nextDisplayMode for FT-04/FT-05, AlarmTask/evaluateTemperature for FT-06/FT-07, MotionTask/StateTask/evaluateSystemState for FT-08–FT-10) but were not individually re-verified with recorded observations in the final simulation session before the submission deadline; this is flagged here explicitly rather than claimed as passed without evidence, consistent with the requirement that a test not be marked PASS without a recorded observation.

5.3 Deliberate fault experiments.

Given the submission deadline, the three required fault experiments (removing a task's blocking delay, assigning an unnecessarily high priority to a frequent task, and removing the serial-output mutex) were not performed and recorded in this cycle. Their expected outcomes, based on the implemented architecture, are noted for completeness: removing SensorTask's vTaskDelayUntil() would let it monopolize CPU time at its priority level, starving DisplayTask (priority 1) of scheduling opportunities; elevating a frequent task like MotionTask beyond its current priority relative to InputTask could delay encoder responsiveness during periods of motion activity; and removing the serialMutex around safePrintf() would allow interleaved output when two tasks log at nearly the same tick, since UART writes are not atomic across multiple task-issued characters.

6. Static Code Analysis

platformio.ini configures `pio check` to run `cppcheck` with ESP-IDF framework packages suppressed, so findings are scoped to this project's own source rather than the underlying SDK. Given the submission deadline, the full findings table was not completed with individual interpretations and corrective actions in this cycle; this is recorded as an open item rather than omitted silently.

7. Engineering Discussion

Trade-offs. Choosing a plain-C implementation over the initial C++ scaffold traded some expressiveness (no enum class, no RAII-style mutex wrappers) for a build that matched the toolchain actually available in the development environment. This was judged the right trade-off given the deadline: a working, simulated, and partially-verified system in C outweighed a more elegant but unverified C++ version.

Debugging. The most time-consuming issue was a non-obvious CMake/git interaction: ESP-IDF's build system calls `git describe` against its own framework installation to stamp a version string, which fails when that installation has no `.git` history of its own, producing a misleading "Configuring incomplete, errors occurred" message that had nothing to do with the actual application code. This was resolved by giving the ESP-IDF build a null Git executable path via `board_build.cmake_extra_args`. A second class of issues involved ESP-IDF struct initializers being treated as hard errors by `-Werror=missing-field-initializers` on this toolchain version, requiring explicit zero-initialization (`= { }`) before setting individual fields.

Problems encountered. The VS Code Wokwi extension's serial monitor produced no output at all for an extended debugging session, despite a successfully built `firmware.bin`. Isolating the cause required reproducing the same firmware on wokwi.com directly (bypassing the VS Code extension entirely), which

confirmed the firmware itself booted and logged correctly — the fault was specific to the local editor integration, not the application. This is recorded as an alternative approach that successfully unblocked verification when the primary tool proved unreliable.

Limitations. As noted in the README, the light-level reading is an uncalibrated raw-ADC-to-percent mapping rather than true lux, and DHT22 timing was verified only in simulation, not on physical hardware.

8. Conclusion

This laboratory produced a working, six-task FreeRTOS application demonstrating explicit task priorities, queue-based and event-group-based inter-task communication, mutex-protected shared output, periodic execution via `vTaskDelayUntil()`, and a centralized ACTIVE/INACTIVE state machine, verified running under Wokwi simulation. The most valuable lesson was architectural: a plain FreeRTOS queue is single-consumer, which is easy to overlook when a design diagram shows one data source branching to two consumers, and catching that early avoided a silent data-starvation bug between the display and alarm subsystems. Given more time, the top priorities would be completing automated pio test execution in this toolchain, recording the remaining functional tests (FT-04–FT-10) and the three fault experiments with actual observations, and running pio check to a fully interpreted findings table.